

# WebRTC & Traffic Shaping

Redes e Serviços

Daniel Corujo, Paulo Salvador, Miguel Matos

7 May 2026

## Foreword

### A: The Problem WebRTC Solves

Every pattern covered so far assumed at least one party had a stable, reachable address. WebRTC targets the harder case: two browser tabs, each behind a NAT router, with no routable address either can dial directly.

Browser A      Router A      Internet      Router B      Browser B  
192.168.1.5:xxx — NAT (82.x.x.x) — NAT (95.y.y.y) — 192.168.1.8:yyy

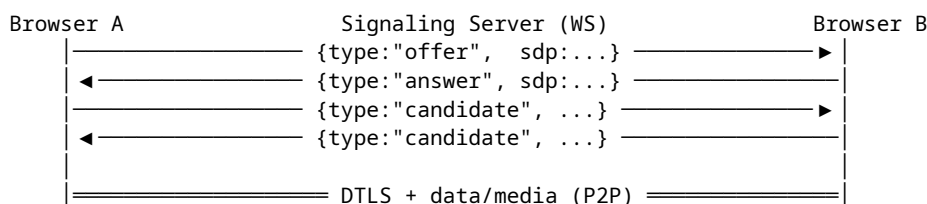
**WebRTC** is the W3C/IETF browser API for real-time peer-to-peer communication. Getting there requires solving three sub-problems:

1. Discovering your own public address: **STUN** (Session Traversal Utilities for NAT)
2. Finding a working path between two peers: **ICE** (Interactive Connectivity Establishment)
3. Exchanging the information needed to attempt those connections: signaling (deliberately left undefined by the standard)

If no direct path is possible even after ICE, a **TURN** relay server acts as a packet forwarder. This lab uses a local STUN server but does not exercise TURN, because both “peers” are on the same machine.

### B: Signaling: The Deliberately Undefined Part

WebRTC does not define how peers first contact each other. `RTCPeerConnection` accepts and emits opaque strings (SDP blobs and ICE candidates) and trusts the application to deliver them out-of-band via a *signaling channel*: any transport that reaches the remote peer before the WebRTC connection exists. In this guide it is a WebSocket server.



The signaling server is only needed during setup. Once the peer-to-peer connection is established, the signaling server is no longer in the data path.

### C: SDP and ICE: What Peers Tell Each Other

Two types of messages flow through the signaling channel.

**SDP** (Session Description Protocol) is a capability declaration, not a request. The offer advertises the supported codecs, media formats, DTLS fingerprint, and ICE credentials. The answer declares what was chosen from the offer. SDP is a line-oriented ASCII format inherited from VoIP infrastructure; it is not designed for human readability. You will see a real SDP blob in Part 1.

The offer/answer exchange follows a strict state machine:

Caller: `createOffer()` → `setLocalDescription(offer)` —[signal]—► `setRemoteDescription(answer)`  
Callee: `setRemoteDescription(offer)` → `createAnswer()` → `setLocalDescription(answer)` —[signal]—►

**ICE** is the process by which both peers discover all the network addresses they might be reachable at, exchange them, and test which pairs actually work. Each address is called a *candidate*. There are three candidate types:

- **host**: a local interface address (e.g., 192.168.1.5:54321)
- **srflx** (server-reflexive): the public IP:port as seen by a STUN server
- **relay**: an address on a TURN relay server, used only as a last resort

Candidates are gathered asynchronously and sent through the signaling channel one by one as they are discovered (*trickle ICE*). Both peers run *STUN binding checks* against every candidate pair and promote the first pair that succeeds to the active path.

## D: RTCDataChannel and the Encryption Mandate

Before any data or media can flow, WebRTC mandates a *DTLS handshake* (Datagram TLS over UDP). This is non-negotiable: the browser enforces it, and the fingerprint in the DTLS certificate must match the fingerprint attribute in the SDP. After the handshake, both sides share a key that secures:

- **RTCDataChannel**: encrypted over DTLS-SCTP. Message-oriented: each `send()` call produces exactly one onmessage event on the receiver. Can be configured as reliable-ordered (TCP-like) or unreliable best-effort (UDP-like).
- **Media tracks**: encrypted over SRTP (Secure RTP). Audio and video frames are carried in RTP packets that are encrypted using keys derived from the DTLS handshake.

This guide starts with DataChannel on parts 1 and 2, while part 3 adds a video track on top of the same connection architecture.

---

## Exercises

### Practical Lab: Real-Time Communication with WebRTC

**Objective:** Establish a WebRTC DataChannel connection between two browser tabs using a WebSocket signaling server; observe SDP blobs and ICE candidate exchange; capture STUN binding requests and DTLS handshake with Wireshark; extend the connection to carry a live video track and identify RTP packets.

All exercises run on the class VM. The signaling server and STUN server run in Docker; the browser pages are served by a local HTTP server (also in Docker).

### Setup

Start the services from the support files directory available on the e-Learning platform:

```
$ docker compose up -d
```

This starts three services:

| Service   | Port       | Purpose                           |
|-----------|------------|-----------------------------------|
| signaling | 8765 (TCP) | WebSocket signaling server        |
| web       | 8080 (TCP) | HTTP server for the browser pages |
| stun      | 3478 (UDP) | Local STUN server (coturn)        |

Verify all three containers are running:

```
$ docker compose ps
```

Open two browser tabs to `http://localhost:8080/client.html`. Keep both tabs visible side by side throughout Parts 1 and 2.

## Part 1: Signaling and the First DataChannel Connection

Before either peer knows the other's network address, they must exchange two things through the signaling server: an SDP offer/answer pair (describing capabilities) and a stream of ICE candidates (describing network addresses). This part walks through each step of that bootstrap sequence.

### 1.1 Reading the SDP Offer

In *Tab A*, open the browser's DevTools (F12) and switch to the *Console* tab. Then click the "Create Offer (*Tab A*)" button. The JavaScript calls `pc.createOffer()` followed by `pc.setLocalDescription(offer)`, and logs the full SDP blob to the console.

Read the SDP output and find the following lines:

- `a=fingerprint:sha-256 . . .`: the DTLS certificate hash that the remote peer will use to authenticate the DTLS handshake
- `a=ice-ufrag: . . .` and `a=ice-pwd: . . .`: the ICE credentials the remote peer will include in STUN binding checks
- `m=application . . .`: the media section describing the DataChannel (not audio or video)

The complete SDP will be 30–50 lines. Notice that it describes what this peer *can* do; it does not contain the remote peer's address, which is not yet known.

**Thinking Question:** The SDP offer is generated before any network contact has been made with the remote peer. It does not contain the remote peer's address and does not describe any agreed-upon configuration. Given this, what is the SDP offer actually *for*, and why must the caller call `setLocalDescription(offer)` before sending it?

### 1.2 Exchanging Offer and Answer Through the Signaling Server

Observe the signaling server log in a separate terminal:

```
$ docker compose logs -f signaling
```

Switch to *Tab B* and click "Accept & Answer (*Tab B*)". *Tab B* calls `setRemoteDescription(offer)`, `createAnswer()`, and `setLocalDescription(answer)`, then sends the answer back. In the Docker log, watch for three message types in sequence: offer, answer, and a burst of candidate messages.

**Thinking Question:** The signaling server in this guide broadcasts every message to every connected client. If a third tab opened `client.html` while two tabs were mid-connection, it would receive the answer and ICE candidates. Why is this problematic in a real application with multiple simultaneous users? What is the minimal change to the signaling protocol that would allow the server to route an offer to a specific peer?

### 1.3 ICE Candidates and the DataChannel Opens

After the SDP exchange, both tabs begin gathering ICE candidates. Watch the "ICE Candidates" panel on each tab; candidates appear asynchronously, each with a `typ` field:

- `typ host`: the machine's local IP address (LAN or loopback)
- `typ srflx`: the public IP:port as observed by the STUN server at `localhost:3478`

Each candidate is forwarded through the signaling server to the remote peer (you will see the candidate messages in the Docker log). Both sides run STUN binding checks against each candidate pair. When a working pair is found, the DataChannel transitions to open.

Type a message in *Tab A*'s input and press Send. The message appears in *Tab B*'s log immediately. Type a reply from *Tab B* and it appears in *Tab A*. The entire path is P2P (on this machine, over loopback); the signaling server has no involvement in this message exchange.

**Thinking Question:** ICE candidates continue arriving and being relayed for several seconds *after* the SDP offer/answer exchange is already complete. The DataChannel only opens after at least one candidate pair has been checked successfully. What does this asynchronous sequence tell you about what "connection setup is complete"?

means in WebRTC, compared to the synchronous connect ( ) call you used in guides 07 and 08?

---

## Part 2: Wireshark: STUN and the DTLS Handshake

With the signaling sequence understood, this part observes what happens at the UDP level: the STUN binding requests that discover the srflx candidates and the DTLS handshake that secures the DataChannel.

### 2.1 STUN Binding Requests

Start a Wireshark capture on the loopback interface (lo) before establishing the connection. Use the following display filters to see packets directed at the STUN server and packets directed at other browser tabs:

```
stun && udp.port == 3478
stun && udp.port != 3478
```

Open a fresh pair of tabs to `http://localhost:8080/client.html` and establish the DataChannel connection as in Part 1. Once the channel opens, stop the capture.

Find a *STUN Binding Request* packet. Expand the STUN layer in Wireshark and locate:

- *Message Type*: 0x0001 (Binding Request)
- *Transaction ID*: a 96-bit random identifier that matches the corresponding Binding Response
- *USERNAME attribute* (sometimes present): the ICE credential pair `local-ufrag:remote-ufrag`

Find the corresponding *STUN Binding Response* (same Transaction ID). Expand the XOR-MAPPED-ADDRESS attribute. This contains the public IP:port that the STUN server observed the request arriving from, which is the source address as seen from outside the NAT. Since both the browser and the STUN server are on the same machine in this lab, this address will be the loopback or LAN address.

In a real deployment, compare this address to the machine's LAN address:

```
$ ip addr show
```

If your machine is behind a NAT router, the XOR-MAPPED-ADDRESS IP will differ from your LAN IP; it will be the router's public IP. This is the srflx candidate.

**Thinking Question:** A STUN server does nothing except receive a UDP packet and send back a response containing the source IP:port it observed. Why does discovering your own public IP:port require sending a packet to an external server? Why can the browser not compute this address locally, the way it knows its own LAN IP?

### 2.2 ICE Connectivity Checks Between Peers

Still using the same Wireshark capture (or a new one), look for STUN messages that are not directed to port 3478. These are the *ICE connectivity checks*: STUN Binding Requests sent directly between peers. Each carries a *PRIORITY* attribute and an *ICE-CONTROLLED* / *ICE-CONTROLLING* attribute identifying the sender's role. The valid pair with the highest priority becomes the active path.

In this lab both peers are on the same machine, so the winning pair is almost certainly a host candidate on the loopback address.

**Thinking Question:** In this lab the host candidate wins, so the STUN server's srflx candidate is never used as the active path. Under what network conditions would the srflx candidate be needed instead? Under what conditions would even the srflx candidate fail, making a relay (TURN) candidate the only option? Why is a TURN server more expensive to operate than a STUN server?

## 2.3 DTLS Handshake

Change the Wireshark display filter to `dtls`. Re-establish the DataChannel connection from Part 1. The DTLS handshake appears as a short burst of UDP packets just before the DataChannel opens.

Identify the following packets:

- *ClientHello*: sent by the peer that initiates the DTLS handshake
- *ServerHello* + *Certificate*: the remote peer responds with its DTLS certificate
- *\*CertificateVerify\** / *Finished*: mutual authentication completes

Expand the Certificate packet and find the certificate's fingerprint. Compare it to the `a=fingerprint:sha-256 ...` line in the SDP offer you read in Part 1.1; they should match. This is how each peer verifies that the DTLS session was established by the same entity that sent the SDP, preventing man-in-the-middle attacks.

After the Finished packets, switch the filter back to `udp` and observe the traffic. All subsequent UDP packets on the DataChannel port are opaque ciphertext; the DataChannel messages you type are no longer visible in Wireshark.

**Thinking Question:** In guide 06, WebSocket messages sent over a plain `ws://` connection were fully readable in Wireshark. DataChannel messages sent after the DTLS handshake are not. Both use IP as the network layer. What is the architectural difference that explains this? Is a WebSocket connection ever encrypted by default, the way WebRTC enforces DTLS?

---

## Part 3: Media Streams over RTP

Parts 1 and 2 established a DataChannel between two browser tabs. The same `RTCPeerConnection` API handles video and audio by adding *media tracks* to the connection before creating the offer. The SDP then includes a negotiated codec, a unique stream identifier (SSRC), and additional parameters for congestion control. Media frames are carried over **RTP**, encrypted with **SRTP** using keys derived from the DTLS handshake.

Open two browser tabs to `http://localhost:8080/media.html`.

### 3.1 Capturing a Video Track

1. In *Tab A*, choose a video source:
  - Click **"1a. Capture Camera"** and grant permission when the browser prompts; or
  - Click **"1b. Upload Video (loop)"** and select a `.webm` file.
2. Verify the local video feed appears in Tab A's left `<video>` element.
3. Repeat in *Tab B*. Both tabs must show a local feed before proceeding.

Both buttons produce a `MediaStream` that WebRTC treats identically. *"1a. Capture Camera"* calls `navigator.mediaDevices.getUserMedia({ video: true, audio: false })`, which prompts for camera permission. *"1b. Upload Video (loop)"* exposes the file as a `MediaStream` via `captureStream()`.

To inspect the active capture parameters, open DevTools (F12) and run in the console:

```
localStream.getVideoTracks()[0].label  
localStream.getVideoTracks()[0].getSettings()
```

`getSettings()` returns the actual values negotiated with the device: `width`, `height`, `frameRate`, and `deviceId`. `label` identifies the physical or virtual device.

### 3.2 How the SDP Changes When a Video Track Is Added

1. In *Tab A*, click **"2. Create Offer (Tab A)"**. The SDP offer is displayed on the page and logged to the console.
2. In *Tab B* (after it also has a video source from 3.1), click **"3. Accept & Answer (Tab B)"**.
3. Verify: remote video from Tab A appears in Tab B's right-hand `<video>` element, and vice versa.
4. In the SDP offer, locate and note the values of these three fields:
  - `a=rtpmap:...`

- `a=ssrc:...`
- `a=rtcp-fb:...`

Before calling `createOffer()`, the JavaScript calls `pc.addTrack(track, localStream)` to attach the video track to the peer connection. This causes the SDP to include an `m=video` section in addition to (or instead of) the `m=application DataChannel` section.

The three fields you located mean:

- `a=rtpmap:<pt> VP8/90000` (or H264, VP9, AV1): maps a payload type number to a codec name and clock rate. The offer lists several candidates; the answer picks one.
- `a=ssrc:<number> ...`: the *SSRC* (Synchronization Source) — a 32-bit identifier that labels this peer's RTP stream. You will recognise this number in Wireshark in section 3.3.
- `a=rtcp-fb:<pt> nack` and `a=rtcp-fb:<pt> goog-remb`: RTCP feedback mechanisms for packet-loss recovery and bandwidth estimation.

Tab B's answer selects one codec from Tab A's offer and creates the reverse video track in the same exchange.

**Thinking Question:** The SDP offer lists five or six video codecs (VP8, VP9, H264, AV1, ...). The answer selects exactly one. Who makes this selection: the browser, the application developer, or the remote peer? Is there any API that allows the application to force a specific codec? Why might a video conferencing application care which codec is chosen?

### 3.3 RTP Packets in Wireshark

Start a Wireshark capture on the loopback interface before establishing the video connection. Once the remote video appears in Tab B, apply the display filter:

`udp`

You will see UDP packets carrying the video stream, but Wireshark may not recognise them as RTP because the port is not a well-known RTP port. To tell Wireshark how to decode them:

1. Right-click on one of the UDP packets from the video stream (a high-rate stream of similarly-sized packets is a good hint).
2. Select *Decode As...* from the context menu.
3. In the dialog, the current row already has the UDP port pre-filled. In the *Current* column on the right, open the drop-down and select *RTP*.
4. Click OK.

Wireshark will now decode all UDP traffic on that port as RTP. Change the display filter to:

`rtp`

RTP packets from the video stream appear continuously at the frame rate of the video (~30 per second). Click on one packet and expand the *Real-Time Transport Protocol* layer. Identify:

| Field           | Meaning                                                                      |
|-----------------|------------------------------------------------------------------------------|
| Payload type    | The number assigned in the SDP <code>a=rtpmap</code> line (e.g., 96 for VP8) |
| Sequence number | Increments by 1 for each packet; gaps indicate packet loss                   |
| Timestamp       | Media clock ticks; used for synchronisation and jitter buffering             |
| SSRC            | Matches the <code>a=ssrc</code> value from the SDP in Part 3.2               |

The payload bytes are opaque: they contain SRTP-encrypted video data and cannot be decoded without the session keys derived from the DTLS handshake.

This is the bottom of the WebRTC protocol stack you have assembled across this guide: IP/UDP carries DTLS (key exchange) and then SRTP-wrapped RTP (media), while the signaling server's WebSocket connection (which closed after setup) sits entirely outside this path.

**Thinking Question:** RTP carries no built-in retransmission mechanism. If a packet carrying a video frame is lost in transit, the receiver's decoder never gets a TCP-style retry request. What options does a video decoder have when a packet is lost, both to recover the content and to signal the problem back to the sender? Why is waiting for retransmission generally not an acceptable strategy for live video?

---

## Part 4: QoS and Traffic Shaping with tc

Linux's `tc` (traffic control) command with the `netem` (Network Emulator) `qdisc` lets you inject delay, packet loss, and rate limits on any interface, including the loopback interface `lo`, through which the two browser tabs are exchanging WebRTC traffic.

Keep the video connection from Part 3 running throughout this part, or re-establish it. Apply each rule only after the WebRTC connection is fully established (signaling is already done; the impairment will not affect the connection setup itself).

### 4.1 Latency and Jitter

Add a fixed delay to all loopback traffic:

```
sudo tc qdisc add dev lo root netem delay 200ms
```

Watch the remote video in Tab B. Observe the lag between Tab A's source and Tab B's display. In Wireshark, note that the RTP **Timestamp** field advances at the normal media clock rate but packets arrive 200 ms later than expected; the jitter buffer must absorb this offset.

Add jitter on top of the delay ( $\pm 50$  ms with a correlation of 25 %):

```
sudo tc qdisc change dev lo root netem delay 100ms 50ms 25%
```

Observe whether the video becomes choppy as jitter exceeds what the jitter buffer can hide.

Restore the interface before the next section:

```
sudo tc qdisc del dev lo root
```

**Thinking Question:** The RTP Timestamp field encodes the media capture time, not the transmission time. Given that, how could a receiver distinguish between a sender that is genuinely slow to produce frames and a network that is adding variable delay? What field or mechanism enables this distinction?

### 4.2 Packet Loss

```
sudo tc qdisc add dev lo root netem loss 5%
```

In Wireshark, watch for gaps in the RTP Sequence Number field; these mark lost packets. Observe the visual effect on the video in Tab B (frame corruption, decoder freeze, or graceful concealment depending on the codec).

Increase the loss rate:

```
sudo tc qdisc change dev lo root netem loss 15%
```

Note the loss level at which the stream becomes unwatchable, and whether the browser recovers when loss stops.

Restore:

```
sudo tc qdisc del dev lo root
```

**Thinking Question:** At high loss rates the browser may send RTCP **NACK** (Negative Acknowledgement) packets requesting retransmission of specific RTP sequence numbers. Add the Wireshark filter `rtcp` and check whether NACKs appear. At what point does retransmission become counterproductive for live video, and why?

### 4.3 Bandwidth Throttling

Cap loopback to 512 kbit/s, well below the typical VP8 stream in this lab:

```
sudo tc qdisc add dev lo root netem rate 512kbit
```

Observe the video quality and frame rate drop. Add the Wireshark filter `rtcp` and watch for **REMB** (Receiver Estimated Maximum Bitrate) packets, which are sent by the browser's congestion controller as bandwidth estimates back to the sender. These correspond to the `a=rtcp-fb goog-remb` attribute you noted in the SDP in Part 3.2.

Reduce further:

```
sudo tc qdisc change dev lo root netem rate 128kbit
```

Restore when done:

```
sudo tc qdisc del dev lo root
```

**Thinking Question:** TCP also reduces its sending rate when the network is congested, using mechanisms like AIMD and slow start. How does the browser's REMB-based approach differ from TCP congestion control, and why is that difference necessary for real-time video?

---

## References

1. W3C. *WebRTC 1.0: Real-Time Communication Between Browsers*. <https://www.w3.org/TR/webrtc/>
2. Alvestrand, H. RFC 8825. *Overview: Real-Time Protocols for Browser-Based Applications*. IETF, 2021.
3. Petit-Huguenin, M. et al. RFC 8489. *Session Traversal Utilities for NAT (STUN)*. IETF, 2020.
4. Keranen, A. et al. RFC 8445. *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal*. IETF, 2018.
5. Rescorla, E. RFC 8827. *WebRTC Security Architecture*. IETF, 2021.
6. Uberti, J. and Jennings, C. RFC 8829. *JavaScript Session Establishment Protocol (JSEP)*. IETF, 2021.
7. MDN Web Docs. *RTCPeerConnection*. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection>
8. MDN Web Docs. *RTCDataChannel*. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/API/RTCDataChannel>
9. MDN Web Docs. *MediaDevices: getUserMedia()*. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>